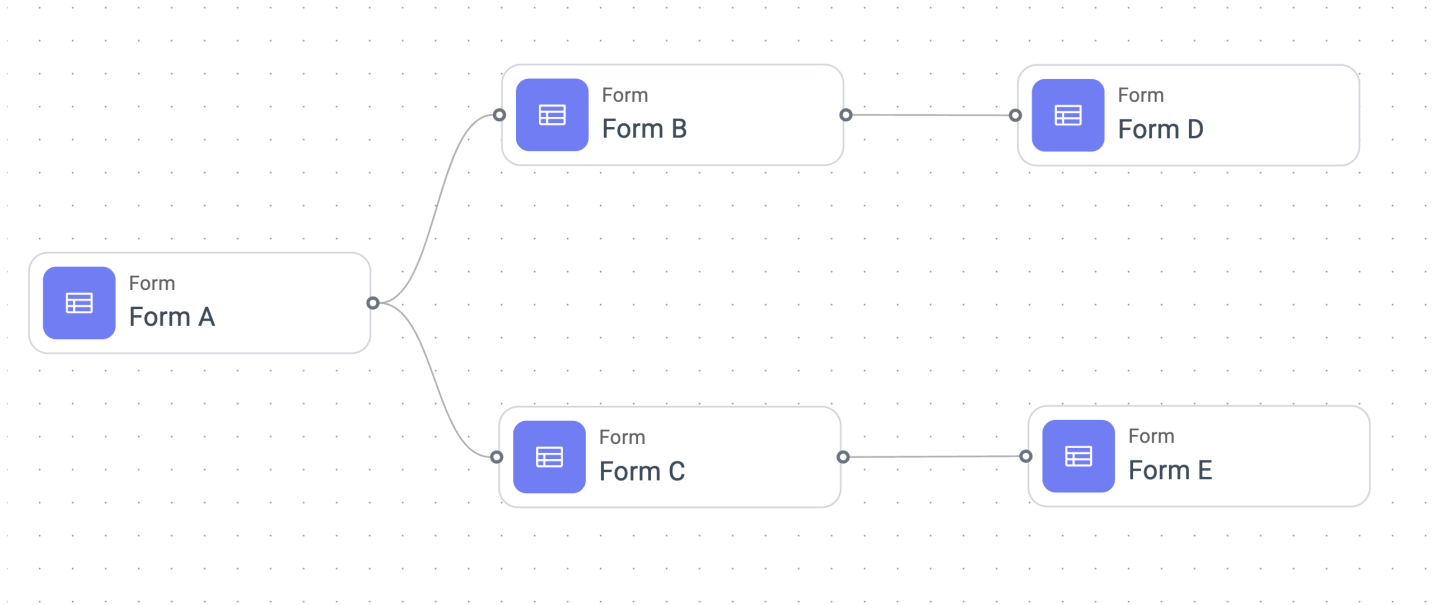


Journey Builder React Coding Challenge

Journey Builder React Coding Challenge

In this challenge, you will reimplement a small portion of an app we’re currently building at Avantos. Avantos has a node-based UI that shows a DAG of forms:



When a form has been submitted, the values from the form fields can be used to prefill the fields of a downstream form. E.g., values from Form A’s fields can be used to prefill Form B’s or Form C’s fields.

First, you will use our docs to hit our `action-blueprint-graph-get` endpoint from a mock server and render a list of forms. **You do not need to render the forms as a node-based UI as in the above screenshot.**

Next, you will implement the prefill UI for Forms. **It doesn’t need to be pretty, but this UI will need to view and edit the prefill mapping.** We show this mapping when a user clicks a node and the mapping looks like this in our UI:

Prefill

Prefill fields for this form

dynamic_checkbox_group

dynamic_object

email: Form A.email

This shows the prefill configuration for three fields on Form D above. The first two fields are called “dynamic_checkbox_group” and “dynamic_object” and they currently have no prefilled data. The last field is called “email” and will be prefilled with the value from Form A’s email field.

Clicking the X button on the far right of the email field clears the prefill configuration for that field. Clicking a field without a configuration opens this modal:

Select data element to map

Available data

Q Search

> Action Properties

> Client Organisation Properties

> Form A

▼ Form B

completed_at

button

dynamic_checkbox_group

dynamic_object

email

id

multi_select

name

notes

CANCEL

SELECT

In this modal, we see 3 types of data that can be used to prefill a form:

- 1. Form fields of forms that Form D directly depends on (Form B)
- 2. Form fields of forms that Form D transitively depends on (Form A)
- 3. Global data (Action Properties and Client Organization Properties)

1 and 2 will require traversing the form DAG. For 3, you can ignore Action Properties and Client Organization Properties and use whatever global data you want.

You should design your code so that any combination of these data sources can be easily used without code changes. Moreover, you should design for easy support of future, new data sources.

Prerequisites

- Use
 - React
 - TypeScript (optional but *strongly* recommended)
 - Create React App, Vite, or Next.js

Rules

SUBMISSION: Your submission should contain:

- 1. A link to your solution as a Github repo.
- 2. AI use is encouraged, but you should understand and own your code. We will be building on it in the live portion.

TIME LIMIT: Send us a link to your Github repository within 4 working/business days from the date you receive this. You may submit your work earlier.

Evaluation Criteria

We will be particularly interested in:

1. Code Organization
 - Clear separation of concerns
 - Thoughtful component hierarchy and composition
2. Extensibility
 - **Does the project have good tests?**
 - Reusable and composable React components
3. Documentation
 - How do I run this locally?
 - How do I extend with new data sources?
4. Code Quality
 - Clean, readable code
 - Reasonable, readable var names
 - Appropriate use of modern React practice